

Transaction Engine — Journal Entry Scenario Engine

Read-only summary. Nothing in the engine was changed to produce this doc.

1. What it is

The **Journal Entry (JE) Scenario Engine** is the app's transaction engine. Its core idea (stated at the top of `je-engine.ts`):

- > The atom is **not** a journal entry. It is a **Scenario**: a real-world event
- > that produces journal entries which vary based on **conditions**.

A **Scenario** declares **condition axes** (e.g. "Sale outcome: gain / loss / break-even") and holds enumerated **variants**, each tagged with the condition values it represents. Fix the conditions *!* you get one **variant** *!* that variant's

journal entry. Everything else — the **ledger (T-accounts)**, the ***financial-statement* effects, **the** trial balance, **and the** $A = L + E$ accounting equation**** — is a **live projection computed from that entry**, never separately authored. "One truth, many views."

The engine is deliberately **pure** (no React, no `fetch`, no Supabase) so its projection outputs can later feed not just the student tool but an auto-rendered video pipeline (Remotion / Motion-Canvas).

2. File paths

Layer	File	Role
Pure engine	<code>src/lib/je-engine.ts</code>	All types + projection functions. No I/O.
Data boundary	<code>src/lib/je-api.ts</code>	The only DB access for JE (reads <code>je_scenarios</code> , builds the browse tree, loads principles + account metadata).

Student route	<code>src/routes/je.tsx</code>	<code>/je</code> — the student-facing viewer (pick a scenario, toggle conditions, watch the projections update). Read-only.
Account metadata	<code>src/lib/ceq-api.ts</code>	<code>fetchChartOfAccounts()</code> !' <code>AccountMeta[]</code> (account_type + normal_balance), reused by the engine.
Types (generated)	<code>src/integrations/supabase/types.ts</code>	<code>je_scenarios</code> is not in the generated types yet — reached via <code>as never</code> / <code>as any</code> casts (same pattern as other post-typegen tables).

Key exported engine functions (all pure): `resolveVariant`

```
, resolveComputationPath
,
validateEntry,
isBalanced
,
deriveLedger
,
deriveStatementEffects
,
deriveEquationEffect,
tracePostingsToStatementLine
.
```

3. Data model

The canonical store: ``je_scenarios`` (one jsonb doc per scenario)

`je-api.ts::ScenarioRow`:

<code>je_scenarios</code>	
<code>id</code>	<code>uuid</code>
<code>slug</code>	<code>text</code> -- stable, unique, e.g. "sell-equipment-cash"
<code>title</code>	<code>text</code> -- "Sell equipment for cash"
<code>doc</code>	<code>jsonb</code> -- the entire <code>ScenarioDoc</code> (see below)
<code>chapter_id</code>	<code>uuid</code> -- FK !' <code>chapters.id</code> (added migration 0025); nullable

The whole scenario is a **single jsonb document**

in `doc`

— intentionally NOT

normalized into many tables ("we are prototyping and will iterate the shape constantly"). The `ScenarioDoc` shape (from `je-engine.ts`):

```

ScenarioDoc {
  slug, title, event, // event = plain-English narrative
  courseFamilies?: string[], // e.g. ["intro2", "ia1"] – optional filter tag
  conceptIds?, principleKeys?, // links to concept/principle spines (optional)
  axes: ConditionAxis[], // declared condition dimensions !' render as toggles
  variants: Variant[], // enumerated; each tagged with its condition values
  // optional v2 presentation flags (default off): isSequence, sequenceGroup,
  hasMemorizationGrid
}

Variant { id, label?, conditions: {axisKey: value}, entries: EntryTemplate[],
  computationPaths? }
EntryTemplate { id, caption?, lines: EngineLine[] }
EngineLine extends JeLine { // JeLine = { account, side, label, tooltip }
  id, why?, trap?, // trap = why a tempting WRONG account/side is wrong
  conceptIds?, principleKeys?,
  amount?: number|null, // null/omitted = "???" (Phase 1 default)
  amountSlotKey? // Phase-2 seam (unused today)
}

```

A "transaction"

in this engine =

one ``je_scenarios`` row

whose

doc

encodes the event, its condition axes, and one entry per variant. Amounts are

optional — Phase 1 shows

???

and reasons about

direction

(**!/'**) only.

How transactions map to chapters / course families

Two independent links, both optional:

1. **Structural** — ``je_scenarios.chapter_id` !' `chapters.id` !' `chapters.course_id` !' `courses`.`

This drives the **browse tree**

```

(
  je-api.ts::fetchJeBrowserTree
): course !'

```

chapter !' scenarios. Empty sibling chapters are included "so Lee can see where

content still needs authoring." Scenarios with no `chapter_id`

fall under a

synthetic **"Unassigned"**

group.

2. **Tag** — ``doc.courseFamilies: string[]``

(e.g.

`["ia2"]`

). A soft filter tag

inside the jsonb; not used for the tree today.

chapters and

courses

are the

existing

tables (migrations 0002 / 0025) —

nothing JE-specific. Account metadata comes from `chart_of_accounts`

via

`ceq-api.fetchChartOfAccounts()`.

4. What exists vs. what's stubbed

Working today:

- Scenario resolution (`resolveVariant`) + computation-path narration.
- All live projections from an entry: `deriveLedger` (T-accounts + net balances when amounts exist), `deriveStatementEffects` (income/BS lines with !/" + cash-flow classification via `INVESTING_ACCOUNTS` / `FINANCING_ACCOUNTS` sets), `deriveEquationEffect` (A=L+E net direction), `isBalanced`, `validateEntry`, `tracePostingsToStatementLine` (reverse highlight).
- The `/je` student viewer + the course!'chapter!'scenario browse tree.
- **4 authored scenarios** total (see §6).

Reserved / stubbed (types exist, engine does not use them):

- **Phase 2 — parameterized numbers:** `AmountGenerator`, `AmountBindings`, `BuildBackwardsPrompt`; the `amountSlotKey` / `resultSlotKey` seams. Not wired.
- **Phase 3 — lifecycle sequences:** `SequenceStep`, `SequenceDoc` (multi-period entries, adjusting/reversing between steps). Not wired; `doc.isSequence` / `sequenceGroup`

only gate placeholder UI sections.

- **CEQ layer:**
`CeqScenarioRef`
— CEQs are a *separate* future layer that will reference scenario/variant/line ids. "Do NOT build CEQ authoring now."
- `hasMemorizationGrid !` placeholder grid only.

5. Related but SEPARATE: `chapter\_journal\_entries` (do not confuse)

There is an older, much larger flat JE bank: `**chapter_journal_entries`
(768

rows)** with `je_lines jsonb`

,
`transaction_label`

, `category\_id` !'

`chapter_je_categories`,
`chapter_id`. It is the

CEQ Resource Bank

per-chapter

JE data. **The Scenario Engine does not read it**

(

`je-api.ts`

never touches it;

`ceq-api.ts` doesn't reference it either). If Ch. 13 content should live in the

Scenario Engine (

`/je`

), it goes in

``je_scenarios``

, not here.

6. Adding "all Ch. 13 (IA2 / ACCY 304) transactions"

Ch. 13 = "Long Term Liabilities" (bonds payable, notes payable, premium/
discount amortization, etc.). It **already exists**

as a

`chapters`

row for

Intermediate Accounting 2.

Where the content lands

- **Table:**
`je_scenarios`
—
one row per transaction
(a
`ScenarioDoc`
in
`doc`
, with

chapter_id
pointed at the Ch. 13
chapters
row.

- **Accounts used** must exist in `chart_of_accounts` (Bonds Payable, Premium/Discount on Bonds Payable, Interest Expense, Cash, etc. — the COA already models `liability_adjunct` for bond premium).
- **No new tables, no code changes** to the engine are needed to add content.

Is there an admin UI to add transactions? No.

- There is **no authoring UI**
`je-api.ts`
is
read-only
(only `select` s);
there is ****no insert**
`/`
`upsert`
`/`
`update`
`/`
`delete`
on
`je_scenarios`
anywhere in
the app, **and**
no seed for it in any migration**. The
`/je`
page even shows
the hint `"No scenarios in this chapter yet — this is where you'd author one"`
but that authoring surface **does not exist**
- Therefore content is **code / DB-defined**
: each scenario's
`doc`
JSON is
hand-authored and **`INSERT`ed directly into `je\_scenarios`**
(via the Supabase
Management API or SQL), setting `chapter_id`
. That's how the current 4
scenarios got there (no migration, no UI):

slug	title	chapter
------	-------	---------

adjust-uneared-revenue	Recognize unearned revenue	3 · Adjusting Entries
adjust-depreciation	Record depreciation	3 · Adjusting Entries
merch-sale	Record a merchandise sale	4 · Merchandising
sell-equipment-cash	Sell equipment for cash	8 · Long Term Assets

Practical steps to add Ch. 13

1. Pick the canonical Ch. 13 chapter row.

& p The
courses
table has

duplicate

rows for IA2 (

ACCY 304

and

IA2

, both "Intermediate

Accounting 2"), so **Ch. 13 "Long Term Liabilities" appears twice**

. Decide

which course/chapter is canonical (or dedupe) and use that `chapters.id` for

all Ch. 13 scenarios, so they group under one node in the browser.

2. For each transaction, author a `ScenarioDoc`

(slug, title, event,

axes

,

`variants[].entries[].lines[]`

with

`account`

/

`side`

/

`label`

+

`why`

/

`trap`

;

`amount`

optional). Set

`doc.courseFamilies = ["ia2"]`

for tagging.

3. **INSERT**

each into

`je_scenarios`

(id, slug, title, doc, chapter_id) via the

Management API / SQL. They appear in `/je`

under Ch. 13 immediately.

4. Confirm every account name used exists in `chart_of_accounts`

(add rows there

first if not).

Bottom line: adding Ch. 13 is a

content-authoring

task (write scenario

JSON, insert rows), not an engineering task — but because there's no admin UI, it currently requires a developer/SQL step per scenario (or a small importer, if that volume warrants building one).